

Convenciones. Se indexa desde 1, con $\text{padre}(i) = \lfloor i/2 \rfloor$, $\text{izq}(i) = 2i$ y $\text{der}(i) = 2i + 1$. Se supone que n es potencia de 2 y que $\log n = \log_2 n$.

Una estimación sencilla puede sugerir que construir un montículo cuesta $\mathcal{O}(n \log n)$, porque se llama a HEAPIFY $\mathcal{O}(n)$ veces y cada llamada cuesta $\mathcal{O}(\log n)$. Explique por qué esta cota no es ajustada y demuestre que la construcción de abajo hacia arriba cuesta $\mathcal{O}(n)$. Puede utilizar $\sum_{h=0}^{\infty} h/2^h = 2$. Justifique también por qué la construcción pertenece a $\Omega(n)$.

Sea $T(n)$ el costo de construir el montículo de abajo hacia arriba, n_h el número de nodos de altura h y $h(v)$ la altura del nodo v .

Costo real de una llamada: HEAPIFY sobre un nodo de altura h baja el elemento a lo más h niveles y hace un número constante de comparaciones e intercambios por nivel, así que cuesta a lo más $c(h+1)$ con $c > 0$ constante, no $c \log n$.

Nodos por altura: en un árbol casi completo los niveles del subárbol de un nodo v están llenos salvo el último, así que si v tiene altura h su subárbol tiene al menos $2^h - 1$ nodos en sus primeros h niveles más uno en el último. Dos nodos distintos de la misma altura tienen subárboles ajenos, de modo que

$$n_h \cdot 2^h \leq n, \quad \text{es decir} \quad n_h \leq \frac{n}{2^h}.$$

Por qué $\mathcal{O}(n \log n)$ no es ajustada: el ciclo recorre $i = n/2, \dots, 1$, o sea que ni siquiera llama a HEAPIFY sobre las $n/2$ hojas. La estimación floja le cobra $\log n$ a cada llamada, pero el número de nodos de altura h decae como 2^{-h} mientras el costo apenas crece como h , y la altura $\log n$ la alcanza un solo nodo, la raíz. La cota es válida y regala un factor $\log n$.

Cota superior: se suma por alturas, no por nodos.

$$\begin{aligned} T(n) &\leq \sum_{h=1}^{\log n} n_h c(h+1) \quad (\text{no hay llamadas con } h=0) \\ &\leq \sum_{h=1}^{\log n} \frac{n}{2^h} c(h+1) \quad (n_h \leq n/2^h) \\ &\leq \sum_{h=1}^{\log n} \frac{n}{2^h} 2ch \quad (h+1 \leq 2h \text{ si } h \geq 1) \\ &= 2cn \sum_{h=1}^{\log n} \frac{h}{2^h} \quad (\text{constantes fuera de la suma}) \\ &\leq 2cn \sum_{h=0}^{\infty} \frac{h}{2^h} \quad (\text{todos los términos son no negativos}) \\ &= 4cn \quad (\text{serie del enunciado}) \end{aligned}$$

La constante $4c$ no depende de n , así que

$$T(n) \in \mathcal{O}(n)$$

Suma de alturas de un árbol perfecto: sea S_H la suma de las alturas de todos los nodos de un árbol binario perfecto de altura H . Ese árbol es una raíz de altura H con dos subárboles perfectos de

altura $H - 1$, de donde $S_H = 2S_{H-1} + H$. Los primeros valores son $S_0 = 0$, $S_1 = 1$, $S_2 = 4$ y $S_3 = 11$, cada uno a distancia $H + 2$ de 2^{H+1} , lo que sugiere

$$S_H = 2^{H+1} - H - 2$$

Caso base: el árbol de altura $H = 0$ es un solo nodo de altura 0, así que $S_0 = 0$, y la fórmula da $2^1 - 0 - 2 = 2 - 2 = 0$.

Hipótesis inductiva: para un $H \geq 1$ fijo se supone que vale en $H - 1$, es decir

$$S_{H-1} = 2^H - (H - 1) - 2.$$

Paso inductivo: se parte de la recurrencia y cada renglón hace una sola manipulación.

$$\begin{aligned} S_H &= 2S_{H-1} + H && \text{(recurrencia del árbol perfecto)} \\ &= 2(2^H - (H - 1) - 2) + H && \text{(hipótesis inductiva)} \\ &= 2^{H+1} - 2(H - 1) - 4 + H && \text{(se distribuye el 2)} \\ &= 2^{H+1} - 2H + 2 - 4 + H && \text{(se distribuye el signo)} \\ &= 2^{H+1} - H - 2 && \text{(se agrupan los términos en H)} \end{aligned}$$

La fórmula vale en $H = 0$ y se hereda de $H - 1$ a H , así que vale para toda $H \geq 0$.

Cierre del argumento lineal: un montículo de n nodos tiene altura $\log n$ y cabe dentro del árbol binario perfecto de esa misma altura: cada nodo conserva su profundidad y su subárbol solo puede crecer, así que su altura ahí es al menos la que tiene en el montículo.

$$\begin{aligned} \sum_v h(v) &\leq S_{\log n} && \text{(el montículo cabe en el árbol perfecto)} \\ &= 2^{\log n + 1} - \log n - 2 && \text{(forma cerrada ya demostrada)} \\ &= 2n - \log n - 2 && (2^{\log n} = n) \\ &< 2n \end{aligned}$$

Como cada llamada cuesta a lo más $2ch(v)$ y solo hay llamadas con $h(v) \geq 1$, resulta $T(n) \leq 2c \sum_v h(v) < 4cn$, la misma cota lineal sin recurrir a la serie infinita.

Cota inferior: el ciclo ejecuta una llamada a HEAPIFY por cada $i = n/2, \dots, 1$, o sea $n/2$ llamadas, y cada una compara $A[i]$ contra al menos uno de sus hijos, con un costo de al menos $c' > 0$. Entonces $T(n) \geq c' n/2$, que es $\Omega(n)$.

Con $T(n) \in \mathcal{O}(n)$ y $T(n) \in \Omega(n)$,

$$T(n) \in \Theta(n)$$